基于 TEE 的高效可验证单次安全聚合协议 OPSA：深度解析与综合评价

王嘉轩

摘要：联邦学习（Federated Learning, FL）——这一去中心化的协作机器学习范式——让多方能够在保护数据隐私的前提下，携手完成模型训练。然而，模型更新的每一轮传输，都可能泄露原始私有数据。安全聚合（Secure Aggregation, SA）协议由此成为保护聚合过程中用户隐私的核心屏障。问题是，现有 SA 协议每次聚合至少需要三轮往返通信，且缺乏验证聚合结果的可靠机制。可验证 SA 虽能弥补这一缺陷，却以高昂的通信成本为代价。基于 TEE 的 SA 方案试图削减往返次数，但 TEE 那有限的物理内存又构成了新的计算瓶颈。本文对 OPSA——一种基于 TEE 的高效可验证单次 SA 协议——进行深度剖析。借助服务端 TEE 的力量，OPSA 让服务器在单次运算中即可完成掩码模型的聚合，将通信次数从三次压缩至一次。为规避 TEE 的内存局限，模型聚合与掩码消除等计算密集任务被巧妙地卸载至 TEE 外部，TEE 内部仅处理共享密钥。在此基础上，本文设计了 KhPRF-OPSA（单层掩码方案）和 POT-OPSA（双层掩码方案）两种协议，各自植入了新型密码学原语。承诺机制与签名机制的整合，使得每个客户端仅需增加 O(1)倍通信开销，便足以确保结果的可验证性。与现有最前沿方案相比，OPSA 在多轮聚合过程中实现了 2 至 10 倍的速度提升，同时牢牢守住结果可验证的底线。本文将在原论文基础上，结合课程所学的密码学与网络安全知识，对 OPSA 协议展开深入拓展分析与评价。

关键词：联邦学习；安全聚合；可验证聚合；可信执行环境；隐私保护

一、研究背景与意义

1.1 联邦学习的发展与隐私挑战

大数据与人工智能技术的狂飙突进，让机器学习模型训练对数据的渴求与日俱增。传统的集中式训练——将散落于各机构、各设备的数据汇聚到中心服务器——不仅带来巨大的通信带宽压力，更引发了严峻的数据隐私与安全问题[1]。数据集中，风险随之集中——这是业界不得不面对的现实。为破解这一困局，McMahan 等人在 2017 年提出了联邦学习（Federated Learning, FL）的概念[2]。其核心思想简明而有力：原始数据原地不动，各参与设备在本地训练模

型，仅将模型更新（梯度或权重）上传至中心服务器进行聚合——以此实现多方协作的模型训练。

联邦学习的范式转变，堪称革命性的。首先，原始数据始终留守本地设备，从根源上斩断了泄露路径。其次，通信对象从原始数据集变为模型参数，带宽需求大幅削减。最后，这一范式让医疗、金融等高度敏感的行业，也能跨越机构壁垒展开协作。然而，随着研究不断深入，学界逐渐发现：联邦学习绝非固若金汤。Zhu 等人[3]提出的深度梯度泄露攻击表明，攻击者能从共享梯度中重构用户的原始训练数据。Fredrikson 等人[4]的模型逆向攻击也证实，通过分析模型输出，可以推测训练数据中的敏感属性。这些发现，无情地揭示了联邦学习中隐私保护的脆弱性——也凸显了在模型聚合过程中实施更强隐私保护措施的紧迫性。

1.2 安全聚合技术的演进

为保护模型聚合过程中的用户隐私，安全聚合（Secure Aggregation, SA）技术应运而生。其核心逻辑是：每个客户端以特定方式对本地模型施以掩码处理，服务器则对这些掩码模型进行聚合，从而恢复出明文形式的全局模型——而服务器在整个过程中，无法窥见任何单个客户端的真实模型更新[5]。

2017 年，Bonawitz 等人迈出了关键一步——提出了首个实用的安全聚合协议 SecAgg[5]。该协议利用秘密共享方案与成对掩码技术，确保即使部分客户端中途退出，全局模型仍可成功恢复。在此基础上，Bell 等人[6]推出 SecAgg+，增强了对恶意客户端的鲁棒性。不过，这些方案更适合单轮联邦学习场景——因为每次聚合都需要客户端重新生成并共享密钥，通信开销不可小觑。

为支持多轮聚合，MicroFedML[7]引入一次性设置阶段，实现密钥复用。Flamingo[8]设计了采用成对掩码的多轮安全聚合协议，而 LERNA[9]则通过单一掩码机制殊途同归。然而，尽管这些方案引入了可复用的设置阶段，每次聚合仍需要三次客户端-服务器通信（如图 1 所示）——在高延迟或受限带宽的场景中，这无疑是一道迈不过去的坎。

有趣的是，Flamingo 中的解密者组和 LERNA 中的委员会，均可视为可信实体。于是，研究者们开始追问一个更根本的问题：TEE 能否替代这些特殊组别？

毕竟，在保持 TEE 内部计算效率的同时实现通信次数最优化——这个方向实在太诱人了。

1.3 OPSA 的研究动机与核心贡献

在 TEE 环境中完整执行 SA 操作，可以优化通信往返过程。但 TEE 的物理内存捉襟见肘——这会在聚合大型模型或处理大量客户端时，制造出显著的计算瓶颈。举例来说，一个包含百万参数的全局模型，大约需要 3.8MB 内存空间（每参数 4 字节）。然而，当客户端数量飙升到数百时，内存需求轻易就突破了 TEE 那 128MB 的物理内存红线。

为破解这一难题，Guan 等人[10]提出了 OPSA（One-Pass Secure Aggregation）协议。其核心贡献可概括为三个方面。第一，构建了一种基于 TEE 的高效可验证多轮单次 SA 框架——借助 TEE 的力量，服务器在单次运算中即可完成掩码模型的聚合，通信往返次数锐减；而 TEE 仅负责密钥处理，其内存消耗与模型规模脱钩，计算瓶颈由此消解。第二，在 OPSA 框架内设计了 KhPRF-OPSA（基于密钥同态伪随机函数的单层掩码协议）和 POT-OPSA（基于代理不经意传输的双层掩码协议），两者均植入了创新的密码学原语。第三，整合承诺机制与签名机制，每个客户端仅需增加常数倍的通信开销，即可确保结果可验证。

本研究的意义何在？OPSA 不仅在通信效率上取得了重要突破，更通过 TEE 与密码学原语的创新融合，为解决联邦学习中隐私保护与效率的平衡难题提供了全新的路径。不过笔者认为，该协议对 TEE 硬件的强依赖是一把双刃剑——它带来了性能提升，却也引入了额外的信任假设。从 Kerckhoffs 原则来看，密码系统的安全性应以密钥——而非硬件——的保密性为基础；OPSA 在这方面的权衡，值得进一步深思。深入剖析 OPSA 协议，对推动安全聚合技术的发展，兼具理论价值与实践意义。

二、理论基础与安全假设

2.1 联邦学习与安全聚合基础

联邦学习的一般场景涉及 n 个客户端和一个服务器。在形式化定义中，每个客户端 u 持有私有数据集 D_u，服务器持有初始全局模型参数 $\theta^{(0)}$。每个训练轮次 $r\in[1,R]$，参与方反复执行以下步骤，直至全局模型 θ 收敛。首先，

服务器将当前全局模型 θ^(r-1)广播至所有参与客户端。每个客户端 u∈U 接收上一轮更新后的全局模型 θ^(r-1)，将其与私有数据集结合，训练本地模型：x_u^(r)=θ^(r-1)-η·∇L_u(θ^(r-1))。其中 η 是学习率，∇L_u 是损失函数梯度。随后，服务器收集个体模型 x_u^(r)，计算聚合模型 z^(r)=Σ_{u∈V} x_u^(r)，更新全局模型 θ^(r)后发回客户端，开启下一轮训练。在此过程中，客户端随时可能退出——退出比例甚至可达参与客户端总数的 30%以上，这对协议的抗掉线能力提出了严苛的考验。

从密码学的视角审视，联邦学习的模型更新过程本质上是一个多方计算（MPC）过程。安全聚合是 MPC 的一个特例：目标是在不揭示各方私有输入的前提下，计算所有输入之和。与传统 MPC 相比，联邦学习中的安全聚合有几个鲜明的特点。参与方数量庞大——可达百万量级。模型维度高耸——可达百万参数级别。网络通信频繁波动——移动设备掉线是家常便饭。计算资源千差万别——不同设备算力悬殊。这些特点意味着，通用 MPC 协议直接套用时，将遭遇严重的性能瓶颈。有意思的是，恰恰是这些问题，催生了专门的高效安全聚合协议设计。

安全聚合（SA）的核心思想，是通过密码学手段保护个体模型更新的机密性。流程是这样的：每个客户端 u 在发送模型更新 x_u 之前，先产生一个掩码 mask_u，然后发送 y_u = x_u + mask_u 给服务器。服务器收集所有客户端发送的掩码模型后，计算 Σ_{u∈V} y_u，并利用掩码之间的抵消关系恢复出 Σ_{u∈V} x_u。为应对客户端退出问题，SA 协议还引入了秘密共享机制——说白了，即使部分客户端退出，服务器仍然可以正确剥离退出客户端的掩码。秘密共享的核心操作是：将每个客户端的掩码密钥拆成多个份额，分发给其他客户端保管；当需要恢复退出客户端的掩码时，从足够数量的其他客户端处收集份额，即可重构掩码密钥。

2.2 可信执行环境（TEE）技术

可信执行环境（TEE）是一种硬件级的安全解决方案——Intel Software Guard Extensions（SGX）和 ARM TrustZone[11]是其中最具代表性的实现。TEE 在处理器内部创建一个被称为「飞地」（Enclave）的隔离安全区域，确保其中执行的代码和数据的机密性与完整性。即便操作系统或虚拟机监视器等特权软件被攻破，也无法染指飞地内部的数据。

TEE 提供的核心安全属性包括：第一，隔离执行——保证飞地内的计算不受外部软件干扰；第二，内存加密——保护飞地内数据的机密性；第三，远程认证（Remote Attestation）——允许远程用户验证飞地中运行的应用程序和硬件平台的真实性[12]。这些特性使 TEE 成为构建可信计算基（Trusted Computing Base, TCB）的理想之选。

但 TEE 也有先天不足：其物理内存容量有限（例如 Intel SGX 的飞地内存上限约为 128MB），且与外部环境的数据交换需要经过性能开销较大的内存加密操作。这些限制，在 OPSA 的设计中都得到了充分的考量与精妙的规避。

2.3 密码学基础原语

OPSA 协议的实现，离不开多种密码学原语的支撑。

OPSA 用伪随机函数（PRF）[13]来生成掩码密钥和随机数。所谓 PRF，就是一个确定性函数：给定随机种子和输入值，其输出在计算上无法与真随机数区分——看似简单，实则强大。

密钥同态伪随机函数（Key-Homomorphic PRF, KhPRF）[14]是一类特殊的 PRF，满足加法同态性质：给定两个不同的密钥 k1 和 k2，对于任意输入 x，有 PRF(k1,x)+PRF(k2,x)=PRF(k1+k2,x)。KhPRF-OPSA 正是利用这一性质，实现了单层掩码的高效聚合。这里有一个值得玩味的地方——数学性质直接转化成了协议效率的提升。

向量 Pedersen 承诺方案（Vector Pedersen Commitment）[15]允许承诺者对一个向量进行承诺，并随后打开承诺以验证其原始值。该方案满足两个核心性质：隐藏性（Hiding）——承诺值不泄露被承诺向量的任何信息；绑定性（Binding）——承诺者不能将同一承诺打开为不同的向量值。

数字签名保证了消息的来源与完整性——在 OPSA 中，TEE 实例和客户端分别对承诺值签名，以此确保聚合结果不可否认。

不经意传输（Oblivious Transfer, OT）及其变体代理不经意传输（Proxy Oblivious Transfer, POT）[16]是 POT-OPSA 协议的基础组件。POT 允许三方（发送者、选择者和代理）参与：发送者持有两个输入，选择者根据其选择比

特获得其中一个正确值，而代理在协助传输的过程中，不获知任何有关选择比特或传输值的信息。

2.4 安全模型与假设

协议基于以下安全模型和假设展开分析。

在网络模型方面，协议假设客户端与服务器之间存在经过认证的信道，确保通信双方的身份真实性。同时假设服务器与所有客户端间存在 P2P 安全信道，用于密钥协商等操作。

在敌手模型方面，OPSA 考虑两种类型的敌手。一是半诚实（Semi-honest）敌手——参与方诚实地执行协议，但不排除试图从协议执行过程中获取额外信息。二是恶意（Malicious）敌手——参与方可以任意偏离协议规范，以破坏协议的安全性。OPSA 的安全性证明在半诚实客户端和恶意服务器的联合模型下进行——客户端可能半诚实（试图获取其他客户端的信息但遵循协议），而服务器则可能是恶意的（可以篡改聚合结果）。

在安全假设方面，OPSA 的安全性建立在以下假设之上：判决性 Diffie-Hellman（DDH）假设及其变体二次判决性 Diffie-Hellman（2ODH）假设；伪随机函数的安全性假设；数字签名方案的存在性不可伪造性（EUF-CMA）；以及承诺方案的隐藏性和绑定性假设。假设虽多，但并不苛刻——每一条都是密码学领域的标准假设。

此外，TEE 本身的安全性假设也是 OPSA 安全模型的重要组成部分。协议假设 TEE 能够提供安全的隔离执行环境，且远程认证机制能够确保用户验证 TEE 中运行程序的正确性。

三、方案设计

3.1 OPSA 总体架构设计

OPSA 的总体设计思想，可以用一句话概括：通过服务端 TEE 机制处理客户端退出，使服务器能在单次运算中完成掩码模型聚合。系统角色被划分为三类：参与联邦学习的多个客户端、执行聚合操作的中央服务器、以及部署在服务器侧的 TEE 可信实例。

协议执行分为两个阶段：一次性设置阶段（One-time Setup Phase）和训练阶段（Training Phase）。在一次性设置阶段，每个客户端 u 与 TEE 实例 Me 之间通过密钥协商协议建立共享密钥 ku,e = KA(sku, pke)，其中 KA 代表非交互式密钥协商协议。同时，客户端之间也建立成对共享密钥 ku,v = KA(sku, pkv)，用于 POT-OPSA 协议中的双层掩码。

在训练阶段的每一轮 r 中，流程如下。首先，每个客户端 u 使用与 TEE 实例 Me 共享的密钥计算掩码 mask_u，并将掩码后的模型 y_u = x_u + mask_u 发送给服务器。然后，服务器识别存活客户端集合 V 和退出客户端集合 D，把这些信息传递给 TEE 实例 Me。TEE 实例根据退出客户端的密钥信息，为服务器恢复出聚合掩码 mask_V。最后，服务器用聚合掩码从存活客户端的掩码模型总和中去除掩码，恢复出聚合模型 $\Sigma_{u\in V} x_u$。

这一设计的关键优势在于：TEE 仅处理密钥操作（密钥恢复和聚合掩码计算），其内存消耗仅与客户端数量相关，而与模型规模无关——从而巧妙地避开了 TEE 物理内存有限带来的计算瓶颈。模型聚合和掩码消除等计算密集型任务，均在 TEE 外部的服务器上完成，充分利用了服务器的计算资源。

3.2 KhPRF-OPSA 单层掩码协议

KhPRF-OPSA 是基于密钥同态伪随机函数（KhPRF）构建的单层掩码安全聚合协议。该协议的核心思想，是利用 KhPRF 的加法同态性质，使多个客户端各自生成的掩码能够在聚合后自动抵消——从而仅需单次掩码操作。

在 KhPRF-OPSA 中，每个客户端 u 计算掩码为：
mask_u = KF(F_1(k_u,e, r), b)
其中，F_1 是一个作用于轮次 r 和客户端 u 与 TEE 实例 Me 之间共享密钥 ku,e 上的安全伪随机函数；KF 是一个作用于临时密钥 k_u' = F_1(ku,e, r)和公共参数 b 上的 1-几乎密钥同态 PRF（1-almost key-homomorphic PRF）。

对于存活客户端集合 $V^(r)$，TEE 计算聚合临时密钥 $k_V^(r) = \Sigma_{u\in V^(r)} F_1(k_e,u, r)$。由于 KhPRF 的几乎密钥同态性质，服务器可以计算聚合掩码为 $KF(k_V^(r), b) + e$，其中$|e_i| < \varepsilon$ 对所有 $i\in[1,m]$成立。最终，聚合模型可以通过以下方式恢复：
$\Sigma_{u\in V^(r)} x_u^(r) = (1/|V|) \cdot [\Sigma_{u\in V^(r)} y_u - KF(k_V^(r),$

b)]

该协议在聚合中产生的误差 e 很小——实践中，通过舍入操作就能得到精确结果。KhPRF-OPSA 的优势在于通信开销极低：每个客户端每轮仅需发送一个掩码模型及其承诺、签名，即可完成安全聚合的全部流程。

四、安全性分析

4.1 恶意服务器模型下的安全性分析

OPSA 的安全性分析聚焦于恶意服务器攻击场景——毕竟，服务器是最有能力篡改聚合结果的参与者。在 OPSA 中，TEE 实例 Me 承担了可信第三方的角色，负责管理密钥并为服务器提供聚合掩码。客户端在发送掩码模型的同时，也提交了承诺和签名，从而为结果验证提供了加密学依据。

协议的安全性依赖于几个精心设计的防御层。第一层：掩码机制确保服务器无法直接获取个体客户端的原始模型更新。即使服务器恶意地尝试从聚合结果中反推个体数据，由于缺乏各客户端的掩码密钥，这种尝试在计算上是不可行的。第二层：承诺机制确保服务器不能篡改聚合结果——任何对结果的修改都会导致承诺验证失败。第三层：签名机制提供了不可否认性——TEE 实例和客户端对关键数据签名，确保各方无法抵赖其操作。

从更宏观的安全视角来看，OPSA 的安全性模型假设 TEE 是可信的——这一假设与 TEE 硬件安全性的实际保障程度直接相关。近年来针对 SGX 的侧信道攻击（如 Plundervolt、Cache Side-Channel 等）不断警示我们：TEE 的物理安全性并非绝对。OPSA 的实际安全性，还需要结合物理安全措施进行综合评估。

4.2 抗掉线安全性分析

客户端退出——这是联邦学习中的常态，而不是意外。OPSA 怎么应对？答案是：借助 TEE 完成退出客户端的密钥恢复。

在 KhPRF-OPSA 中，当客户端 v 在轮次 r 退出时，TEE 实例 Me 存储了与客户端 v 共享的密钥 $k_{e,v} = KA(ske, pkv)$。TEE 利用这些密钥重新计算退出客户端的掩码分量，并将其纳入聚合掩码的计算中——从而确保退出客户端的掩码不会影响最终聚合结果的正确性。

在 POT-OPSA 中，TEE 实例还需恢复退出客户端的个体密钥分量 $k_{e,v}$ 和成对密钥分量$\{k_{e,v,u}\}$（用于与其他存活客户端的成对掩码）。通过将这些密钥分量纳入聚合掩码计算，服务器能够正确地从总掩码模型中去除所有退出客户端引入的掩码。

协议的抗掉线能力满足以下条件：当存活客户端数量$|V| \geq \lceil(1-\delta)N\rceil$且$(1-\delta) \geq \alpha$ 时（其中 α 为求和理想功能的最小存活比例），聚合可以成功完成。这一条件确保了 OPSA 能够在合理的客户端退出比例下正常运作——不掉链子，不改结果。

4.3 结果真实性与完整性分析

结果验证机制针对恶意服务器的两种篡改行为提供了坚实保护。假设恶意服务器 A 试图输出一个被篡改的全局模型 $z_A = z_H + \tilde{x}_A + e$（其中 z_H 是诚实存活客户端的真实聚合结果，$\tilde{x}_A$ 是被篡改的个体模型，e 是引入的误差向量），半诚实客户端可能接受 z_A 的情况有两种。

第一种情况：服务器能够基于相同的 com 和 rand 生成 z_A。这要求服务器找到一个 x'使得 $Commit(x', rand) = com$——而这直接违反了向量 Pedersen 承诺方案的绑定性，在计算上是不可行的。

第二种情况：服务器能够为新的承诺 com'和随机数 rand'伪造 TEE 实例 Me 的签名。这要求攻击者能够在不知道签名私钥 ske 的情况下生成有效签名——而这违反了 EUF-CMA 安全签名方案的安全性假设。数学上不可行，那就安全了。

所以，结论很清晰：在承诺方案的绑定性假设和签名方案的 EUF-CMA 安全性假设下，OPSA 的验证机制能够有效检测恶意服务器的篡改行为。说白了，服务器想动手脚？没那么容易。

4.4 混合安全证明框架

安全性证明采用标准的混合论证（Hybrid Argument）方法。该证明通过一系列不可区分的混合实验，将协议的实际执行逐步归约到理想功能的安全模拟——这是密码学证明中的经典套路，但细节之处见真章。

以 KhPRF-OPSA 为例，安全性证明跨越五个混合步骤。Hyb0 是实际协议执行中所有参与方的联合视图。Hyb1 中，模拟器掌握所有诚实参与方的秘密信息，

并模拟协议执行。Hyb2 将共享密钥替换为随机密钥——基于 2ODH 假设。Hyb3 将掩码模型替换为均匀随机采样值。Hyb4 将 TEE 输出的密钥和聚合掩码替换为模拟值。Hyb5 将理想功能的输出设为真实的聚合模型。

通过这五个混合步骤的层层递进，可以证明：实际执行中敌手的联合视图，在计算上与模拟器的输出不可区分——从而证明了 OPSA 协议在恶意服务器模型下的安全性。类似的证明框架也适用于 POT-OPSA 协议，区别在于需要额外处理成对密钥的安全替换。

五、比较分析与应用

5.1 与现有方案的通信效率对比

通信效率方面，OPSA 相比现有方案具有压倒性优势。传统安全聚合协议（如 SecAgg）每轮需要三次往返通信：第一次广播公钥，第二次发送掩码模型和秘密共享份额，第三次发送掩码密钥。即便经过优化的多轮方案（如 Flamingo 和 LERNA），也未能打破这一通信模式。

通过利用服务端 TEE 处理客户端退出，OPSA 将所需通信次数锐减至单次往返。在 KhPRF-OPSA 中，客户端在一轮训练中仅需发送一次掩码模型（附带承诺和签名），服务器在 TEE 协助下即可完成聚合并返回结果。这种单次通信模式，在处理大型模型或高延迟网络环境时，优势尤为显著。想想看：本来要等三趟，现在一趟搞定。若网络往返时间为 RTT，安全聚合的通信延迟即为协议所需通信次数与 RTT 的乘积。在 RTT 为 100ms 的典型广域网环境中，传统方案的每轮聚合延迟为 300ms，而 OPSA 仅需 100ms——每轮节省 200ms。当训练需要数千轮聚合时，累计的时间节省可达数十分钟甚至数小时。

从通信复杂度的理论分析来看，设 n 为客户端数量，m 为模型参数数量，则 SecAgg 的单轮通信复杂度为 $O(n^2 + n \cdot m)$，而 OPSA 的通信复杂度为 $O(n + n \cdot m)$——消除了 n^2 项。在大规模客户端场景下，这一差异是决定性的。当客户端数量 n 达到 1000 时，SecAgg 的通信开销中 n^2 项占据主导地位，使得协议几乎无法实用；而 OPSA 的线性复杂度，则能够优雅地扩展到大规模场景。

5.2 计算开销与 TEE 内存效率对比

在 TEE 内存效率方面，OPSA 的设计展现了令人印象深刻的创新性。现有 TEE-based 方案（如 SEAR[17]）尝试通过在 TEE 内部逐层处理模型参数来缓解

内存瓶颈——但这并未从根本上减轻 TEE 的整体计算负担。SEAR 采用了逐层聚合（Layer-wise Aggregation）策略，每次只在 TEE 内部处理模型的一层参数。然而，当模型中存在包含大量参数的层（如全连接层具有数千万参数）时，SEAR 仍然面临 TEE 内存不足的尴尬局面。

OPSA 另辟蹊径：将计算密集型任务（模型聚合和掩码消除）卸载至 TEE 外部，仅在 TEE 内部处理密钥操作。这使得 TEE 的内存消耗与模型规模彻底解耦。这一设计理念，与「最小特权原则」有异曲同工之妙——只让 TEE 做它最擅长的事（密钥管理），而非什么都往里塞。具体而言，TEE 仅需存储每个客户端的密钥信息（约 32 字节/客户端），而不需存储整个模型。对于包含百万参数的大型模型和数百个客户端的场景，TEE 的内存需求从数百 MB 降至数 KB——差距悬殊，令人咋舌。内存占用的骤降不仅解决了 TEE 的空间限制问题，还减少了内存加密/解密操作的开销，形成正向循环。

在计算效率方面，实验结果表明 OPSA 在多轮聚合过程中实现了 2 至 10 倍的速度提升。这一性能飞跃主要源于两个方面：一是通信次数的减少降低了网络延迟的影响——在高延迟网络环境中表现尤为突出；二是 TEE 仅承担轻量级密钥操作，避免了大型模型在 TEE 内部处理带来的性能瓶颈。实验数据表明，在 100 个客户端、模型参数量为百万级别的场景下，KhPRF-OPSA 的聚合时间约为 SecAgg 的五分之一，POT-OPSA 约为 SecAgg 的三分之一。POT-OPSA 的聚合时间略高于 KhPRF-OPSA——这是双层掩码机制引入额外计算开销的代价，但相比传统方案，仍有显著的性能提升。

5.3 应用场景分析

OPSA 协议适用于多种联邦学习应用场景——在以下几个领域，它的优势尤为突出。

首先是跨设备联邦学习。以输入法数据联邦学习模型训练为例：当涉及大量移动设备或 IoT 设备时，网络连接不稳定、设备频繁掉线是常态。OPSA 的高效抗掉线机制和单次通信模式，让这些场景下的安全聚合变得真正实用。在 Google 的 Gboard 输入法预测模型训练中，参与的移动设备数量可达数百万，设备掉线率高达 5%以上。传统的 SecAgg 等方案在此类场景中通信开销巨大，

而 OPSA 通过单次通信完成聚合，且 TEE 高效处理掉线问题——大幅提升了系统的实用性。

其次是高延迟网络环境。在地理分布广泛的跨区域联邦学习场景中（如跨国医疗机构协作），网络延迟可能成为性能瓶颈。OPSA 将每次聚合所需的通信往返从三次减少到一次，显著降低了延迟对训练效率的拖累。例如，在跨大西洋的联邦学习场景中，网络 RTT 可达 150ms 以上——OPSA 单轮可节省 300ms 的通信时间，累计训练时间缩短超过 66%。

第三是大型模型训练场景。在训练深度神经网络等大型模型时，OPSA 通过将模型聚合操作从 TEE 内部卸载到外部服务器，避免了对 TEE 内存资源的过度消耗，使得大型模型（如 ResNet-50、BERT 等）的安全聚合成为可能。对于包含数千万乃至上亿参数的现代深度学习模型，传统基于 TEE 的方案根本无法在 128MB 的飞地内存中完成聚合——而 OPSA 完全不受模型规模的限制。架构对了，问题就解决了一大半。

第四是合规场景——那些需要结果审计的场景。在金融监管、医疗数据等需要审计和追溯的应用中，OPSA 的验证机制能够提供聚合结果的完整性保证，满足合规性要求。例如，在医疗影像分析的联邦学习场景中，HIPAA 等健康数据隐私法规要求对数据处理过程进行审计——OPSA 的承诺和签名机制为审计提供了不可否认的证据，确保了数据处理过程的完整性和可追溯性。

OPSA 同样适用于智能物联网中的分布式模型训练、自动驾驶汽车的车队学习、智能家居设备的协同学习等新兴场景。随着边缘计算和物联网技术的快速演进，联邦学习的应用版图将持续扩张——OPSA 作为高效可验证的安全聚合方案，在这些新兴场景中拥有广阔前景。

六、总结

本文对 OPSA——一种基于 TEE 的高效可验证单次安全聚合协议——进行了全面而深入的研究与分析。OPSA 将客户端退出管理委托给服务端 TEE，设计了 KhPRF-OPSA 和 POT-OPSA 两种掩码协议。它将通信往返次数从三次压缩到单次，并通过承诺和签名机制实现了结果可验证性。总的来说，OPSA 在通信效率、计算效率和安全性之间，取得了令人印象深刻的平衡。

核心创新体现在三个层面。架构层面：模型聚合和掩码消除被卸载至 TEE 外部，从根本上解决了 TEE 内存瓶颈，使内存消耗与模型规模解耦。协议层面：利用 KhPRF 和 POT 等密码学原语，实现了高效且安全的单层和双层掩码聚合。验证层面：设计了轻量级的结果验证机制——仅以 O(1)的额外通信开销，换来结果可验证性，填补了现有协议在结果验证方面的空白。

回顾 OPSA 协议的设计，可以看到密码学与网络安全课程中多个核心概念的深度融合：安全多方计算思想为安全聚合提供了理论基础；伪随机函数和密钥协商协议为掩码生成提供了密码学工具；承诺方案和数字签名机制为结果验证提供了安全保障；TEE 可信硬件为协议的高效执行提供了物理根基。这种「密码学原语 + 可信硬件」的协同设计理念，代表了现代安全协议设计的一个重要发展趋势。

放眼更广阔的视野，OPSA 代表了安全聚合技术的一个重要演进方向——通过硬件辅助安全技术与密码学原语的协同设计，在保持安全性的同时大幅提升性能。随着 TEE 技术的成熟与普及（如 Intel SGX 的广泛应用），OPSA 等基于 TEE 的安全方案有望在联邦学习实践中发挥越来越重要的作用。然而，也需要清醒地认识到：OPSA 对 TEE 硬件安全性的依赖是其潜在的安全弱点——未来的研究需要在放松硬件安全假设方面进行进一步探索。

未来的研究方向包括：将 OPSA 扩展到拜占庭鲁棒的安全聚合场景，使协议能够抵御隐私攻击和模型投毒攻击；探索更高效的远程认证机制，降低 TEE 认证的开销；将 OPSA 与其他隐私保护技术（如差分隐私）结合，构建更全面的联邦学习隐私保护框架；以及在真实大规模联邦学习系统中部署和评估 OPSA 的性能表现——为其实际应用提供坚实的经验基础。

参考文献

[1] Kairouz P, McMahan H B, Avent B, et al. Advances and open problems in federated learning[J]. Foundations and Trends in Machine Learning, 2021, 14(1-2): 1-210.

[2] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Artificial Intelligence and Statistics (AISTATS). PMLR, 2017: 1273-1282.

[3] Zhu L, Liu Z, Han S. Deep leakage from gradients[C]//Advances in Neural Information Processing Systems (NeurIPS), 2019: 14774-14784.

[4] Fredrikson M, Jha S, Ristenpart T. Model inversion attacks that exploit confidence information and basic countermeasures[C]//Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2015: 1322-1333.

[5] Bonawitz K, Ivanov V, Kreuter B, et al. Practical secure aggregation for privacy-preserving machine learning[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, 2017: 1175-1191.

[6] Bell J H, Bonawitz K A, Gascón A, et al. Secure aggregation with malicious clients[C]//29th USENIX Security Symposium. USENIX Association, 2020: 1277-1294.

[7] Guo Y, Polychroniadou A, Shi E, Byrd D, Balch T. MicroFedML: Privacy-preserving federated learning for small-weight scenarios[J]. Cryptology ePrint Archive, 2022.

[8] Ma Y, Woods J, Angel S, Polychroniadou A, Rabin T. Flamingo: Multi-round single-server secure aggregation with applications to private federated learning[C]//2023 IEEE Symposium on Security and Privacy (SP). IEEE, 2023: 977-995.

[9] Li H, Lin H, Polychroniadou A, Tessaro S. LERNA: Secure single-server aggregation via key-homomorphic masking[C]//Advances in Cryptology (CRYPTO). Springer, 2023: 289-318.

[10] Guan Z, Zhao Y, Wan Z, Han J. OPSA: Efficient and verifiable one-pass secure aggregation with TEE for federated learning[C]//IEEE Symposium on Security and Privacy (S&P). IEEE, 2025.

[11] Will N C, Maziero C A. Intel software guard extensions: A survey[J]. ACM Computing Surveys, 2023, 55(8): 1-37.

[12] Pass R, Shi E, Tramer F. Formal abstractions for attested execution secure processors[C]//Proceedings of the 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Springer, 2017: 260-289.

[13] Banerjee A, Peikert C, Rosen A. Pseudorandom functions and lattices[C]//Proceedings of the 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Springer, 2012: 719-737.

[14] Boneh D, Lewi K, Montgomery H, Raghunathan A. Key homomorphic PRFs and their applications[C]//Proceedings of the 33rd Annual International Cryptology Conference (CRYPTO). Springer, 2013: 410-428.

[15] Pedersen T P. Non-interactive and information-theoretic secure verifiable secret sharing[C]//Proceedings of the 11th Annual International Cryptology Conference (CRYPTO). Springer, 1991: 129-140.

[16] Choi S G, Katz J, Kumaresan R, Cid C. Multi-client non-interactive verifiable computation[C]//Theory of Cryptography Conference (TCC). Springer, 2013: 499-518.

[17] Zhao L, Jiang J, Feng B, et al. SEAR: Secure and efficient aggregation for Byzantine-robust federated learning[J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(5): 3329-3344.

[18] Van Schaik S, Seto A, Yurek T, et al. SoK: SGX.Fail: How often do TEEs fail?[C]//IEEE Symposium on Security and Privacy (S&P). IEEE, 2022.

[19] Jia Y, Liu S, Wang W, et al. HyperEnclave: An open and cross-platform trusted execution environment[C]//2022 USENIX Annual Technical Conference (USENIX ATC). USENIX Association, 2022: 437-454.

[20] Toffalini F, Payer M, Zhou J, Cavallaro L. Designing a provenance analysis for SGX enclaves[C]//Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID). 2023.